

BuildWatch — описание решения для ЛЦТ 2026

Версия: 27 сентября 2026 года. Задача Департамента градостроительной политики Москвы: автоматизированный поиск отклонений на строительных площадках по снимкам камер и календарному плану.

1. Концепция и границы прототипа

BuildWatch связывает объект, датированный снимок, распознанную технику и действующий на дату снимка этап работ. Правила выдают сигнал инспектору, если обязательная техника не обнаружена или наблюдаемый класс не соответствует этапу. В интерфейсе доступны портфель объектов, карточка площадки, план, снимки с рамками детекции, предупреждения и отчёт. Сигнал требует проверки человеком: один ракурс не доказывает отсутствие техники на всей площадке или нарушение сроков.

Справочник ЛТЦ содержит виды работ, но не даты. Календарь задаётся пользователем отдельно или импортируется из CSV/XLSX с колонками «Этап», «Дата начала», «Дата окончания». При импорте сначала показываются ошибки и предпросмотр; запись происходит после подтверждения.

2. Архитектура и поток данных

Пользователь/камера → снимок + дата → FastAPI → SQLite/PostgreSQL

↓

очередь detection_jobs

↓

worker → YOLO → detections

↓

дата снимка + stages → rules → warnings

↓

Next.js: карточка, проверка, отчёт

API принимает снимок, создаёт задание и сохраняет его состояние. Worker запускает локальный детектор, проверяет формат и координаты результата, затем атомарно сохраняет детекции и пересчитывает правила. Неудачный инференс получает статус **failed**, а не трактуется как отсутствие техники. Перезапущенные зависшие задания возвращаются в очередь. Сводка и объяснение правила доступны в UI; инспектор может подтвердить или отклонить сигнал. Публичный репозиторий использует один синтетический демо-объект; кадры и таблицы организатора туда не входят.

Основные компоненты: `backend/app.py` — HTTP API; `backend/db.py` — схема и доступ к БД; `backend/detection_service.py` — проверка и финализация детекций; `backend/rules.py` — сопоставление с планом; `scripts/worker.py` — очередь; `scripts/detect_equipment_single.py` — инференс; `frontend/` — Next.js. Демоданные создаёт `backend/demo_projects.py`.

3. Модель данных

Таблица · Значимые поля · Назначение

`objects` · `id`, `name`, `type`, `district`, `address` · Строительные объекты

`snapshots` · `object_id`, `captured_at`, `status`, `filename`, `src` · Снимки и состояние обработки

`detections` · `snapshot_id`, `model`, `label`, `score`, `x1...y2` · Классы и рамки объектов

`stages` · `object_id`, `position`, `kind`, `name`, `date_from`, `date_to`, `status` · Календарные этапы

`warnings` · `snapshot_id`, `rule`, `severity`, `why`, `source`, `status` · Проверяемые отклонения

`detection_jobs` · `snapshot_id`, `model`, `status`, `attempts`, `error` · Очередь инференса

`catalog` · `code`, `name`, `applies` · Виды работ ЛТЦ

`users`, `audit_log` · Пользователь, роль, действие, время · Доступ и аудит

В демо используется SQLite; схема PostgreSQL хранится в `migrations/001_production.sql`. Кадры хранятся как файлы, БД содержит их пути. Привязка снимка к этапу выполняется по `captured_at` внутри интервала `date_from-date_to`; если совпадения нет, используется этап со статусом `current`. Это резервное поведение следует учитывать при проверке неточного плана.

4. Детектор, данные и правила

В живом контуре настроен один обученный детектор YOLO (`equipment`), а старые имена моделей перенаправляются на него для совместимости с заданиями. Веса хранятся локально в `weights/equipment_v6_m1280_best.pt` и не включаются в публичный Git: их происхождение и право распространения требуется отдельно подтвердить. Без весов доступны интерфейс и синтетический пример с заранее заданной детекцией, новая детекция недоступна. Модель v11 находится в экспериментальном обучении и в прототип не подключена.

Целевые восемь классов ТЗ: самосвал, экскаватор, каток, кран-манипулятор, бетоносмеситель, бульдозер, грузовик, автокран. Внутренняя онтология также содержит вспомогательные классы, например башенный кран, погрузчик и буровую установку. Наличие класса в онтологии не означает одинаковое качество его распознавания. Примерные снимки ДГП не размечены как официальный тестовый набор. Исследовательские наборы и правила происхождения описаны в `docs/external-training-data.md`, `docs/mocs-dataset.md` и `docs/openimages-training-data.md`.

Проверка на MOCS validation (4 000 изображений) зафиксирована в README; она относится к отдельному исследовательскому детектору Thalos и не является оценкой текущего `equipment`. Для ранней версии `equipment_v2` был получен holdout mAP@50 = 0,786 на внешней выборке. Этот результат не подтверждает качество на камерах Москвы: состав внешнего holdout и доменный сдвиг ограничивают вывод. Текущие веса v6 требуют собственной замороженной проверки по всем восьми классам, в том числе на мелких объектах и московских кадрах.

Правила работают с завершённой детекцией и активным этапом:

Код · Условие · Результат

R-01 · Класс техники не разрешён для этапа · Сигнал о несоответствии

R-02 · Низкая уверенность или конфликт классов на рамке · Ручная проверка

R-03 · Не обнаружена обязательная для этапа группа техники · Сигнал о риске снижения темпа

R-07 · Крановый класс обнаружен на этапе фасадных работ · Специализированный сигнал

Порог сохранения рамок и порог правил различаются (0.15 и 0.35); низкая уверенность помечается при `score < 0.5`. Для котлована настроены экскаватор и самосвал. Для подготовки

территории, каркаса и кровли действуют другие наборы; для фасада обязательный список пуст. Категория «нет техники» не вычисляется по кадру в состоянии **new**, **processing** или **failed**.

5. Сборка и запуск

Локально в Windows

1. Установить Python 3.12 и Node.js 20+, затем `pip install -r requirements.txt` и `npm --prefix frontend ci`.
2. Для новой детекции отдельно установить совместимые PyTorch и Ultralytics и положить разрешённые к использованию веса в `weights/equipment_v6_m1280_best.pt`.
3. Выполнить `python scripts/seed_public_demo.py` при первом запуске. Команды `python backend/seed.py` и `python backend/demo_projects.py` предназначены для локального набора организатора при наличии его файлов.
4. Запустить `powershell -ExecutionPolicy Bypass -File scripts/start-buildwatch.ps1`. Интерфейс: `http://127.0.0.1:8700`, API: `http://127.0.0.1:8600/api/health`.

В Linux/WSL три процесса можно запустить отдельно: `cd backend && ../.venv/bin/uvicorn app:app --host 127.0.0.1 --port 8600`, `../.venv/bin/python scripts/worker.py` и `npm --prefix frontend run dev -- --port 8700`. Для детекции нужны Linux-совместимые PyTorch, Ultralytics и локальные веса. Команды выполняются из корня репозитория, кроме API.

Docker Compose

`docker compose up --build` запускает API, worker и frontend; при отсутствии материалов организатора создаётся синтетический пример. Для новых детекций требуется локальный каталог `weights/` с файлами модели. Если используется ИИ-помощник, `BUILDWATCH_ZEN_KEY` передаётся только серверу через переменную среды. Не помещайте ключ в Git. Docker-сборка CPU тяжёлая из-за PyTorch.

Проверки: `../.venv/bin/python -m unittest discover -s tests -v`, `npm --prefix frontend run build`, затем `/api/health` и сквозная загрузка снимка через UI. Качество CV измеряется отдельно; прохождение unit-тестов его не доказывает.

6. Демонстрационный сценарий

1. Открыть синтетический объект с этапом «Устройство котлована» либо локальный портфель при наличии данных организатора.
2. Показать датированный снимок и рамки распознанной техники; при наличии локальных весов загрузить новый снимок и дождаться выполнения задания.
3. Показать выбранный по дате этап и ожидаемые классы.
4. Открыть сигнал R-03 об отсутствующем самосвале либо R-01/R-07 о лишней технике; показать объяснение и источник.
5. Подтвердить или отклонить сигнал и сформировать отчёт объекта.

7. Установка камер и развитие

Камера должна перекрывать рабочую зону и маршруты техники, сохранять время съёмки и идентификатор площадки. Желательны несколько фиксированных ракурсов без сильных перекрытий, достаточное разрешение для удалённой техники, контроль ночного освещения и осадков, а также периодическая проверка чистоты объектива. Прежде чем трактовать отсутствие класса как отклонение, нужно проверить видимость соответствующей зоны и время

относительно графика. На перспективу: многокамерное объединение, калибровка по площадке, отслеживание техники во времени, SAHI для мелких объектов, проверка качества камеры и оценка на размеченном московском наборе.

8. Соответствие ТЗ и ограничения

Прототип реализует четыре обязательные функции ТЗ: детекцию, правила «этап → техника», интерфейс и сигналы о несоответствии. Презентация показывает концепцию, архитектуру, поток «снимок → этап», экран прототипа, рекомендации для камер и план развития.

Документация описывает данные, код и запуск. Ограничения: веса не публикуются автоматически; московские кадры не образуют независимый размеченный test-set; один снимок может не видеть нужную технику; экспериментальные метрики внешних наборов нельзя считать качеством эксплуатации.